

AA-Arena: Can Coding Agents Build Game Agents for Real-World Adversarial Games?

Kaisen Yang^{1,*,\dagger}, Qingle Liu^{1,*,\dagger}, Kejin Wang^{1,*}, Yicheng Zhao^{1,*}, Jieming Li^{1,*}, Shenghan Zheng^{1,*}, Ruize Yang^{1,*}, Bojun Yang^{1,*}, Heng Gong¹, Xiang Gao¹, Lanyue Zhang¹, Kaiyu Zhong¹, Zhuo Liu¹, Shaoxuan Li¹, Chengxi Li¹, Yong Yan¹, Weixuan Zhang¹, Situ Wang¹, Youjie Zheng¹, Sihan Zhao¹, Shengyuan Wang^{1,2}, Huan-ang Gao¹, Jiazheng Xu¹, Xiaohui Xie^{1,\dagger}, Wentao Han^{1,\dagger}

¹ Department of Computer Science and Technology, Tsinghua University

² College of AI, Tsinghua University

* Core Contributor; ^{\dagger} Project Lead; ^{\ddagger} Corresponding Author. *Author details: TBD.*

AI for adversarial games has progressed from heuristic search, exemplified by Deep Blue and Monte Carlo tree search, to reinforcement learning and self-play. Yet adapting efficiently to diverse, changing opponents remains challenging. Building on heuristic learning (HL), we formalize Adversarial Heuristic Learning (AHL), where a coding agent interprets game feedback and revises executable policies while its model weights remain fixed. We introduce AA-Arena (Agents for Agents Arena), comprising 12 real-world competition games and archived human opponents, and evaluate 5 coding models by their best Elo and rank under separate match and full-pool evaluation budgets. Generated policies top some human ladders, but additional interaction does not eliminate improvement bottlenecks in complex games. Ablations highlight the roles of opponent selection, dense replays, and interaction batch size. AHL learns from both on-policy feedback and off-policy replays, demonstrating its potential to improve strategies through both direct interaction and other players' experience.

Contact: yks23@mails.tsinghua.edu.cn, lql24@mails.tsinghua.edu.cn

Project page: [\[TODO: project page URL not supplied\]](#)

Code: [\[TODO: code base URL not supplied\]](#)



1 Introduction

Games have long served as a testing ground for artificial intelligence, with responding to other agents' strategies a central challenge throughout this development. Early systems relied on hand-crafted heuristics, evaluation functions, and tree search; reinforcement learning and self-play subsequently advanced experience-driven policy optimization (Campbell et al., 2002; Coulom, 2007; Silver et al., 2018). Across these approaches, opponents form part of each player's effective environment: different opponents induce distinct, sometimes conflicting strategic modes, and changes in their behaviour shift the learning target. Adapting to these changes under limited interaction while maintaining performance across opponents remains a challenge for continual learning (Hernandez-Leal et al., 2019).

Meanwhile, coding agents have progressed from generating snippets to navigating repositories, executing tests, and revising implementations (Yang et al., 2024). Related computer-use agents extend interaction to graphical interfaces (Xie et al., 2024), while work on agent design and recursive program improvement explores updating the software that supports intelligent behaviour (Hu et al., 2025c; Zelikman et al., 2024). These developments connect several capabilities: *exploration* to acquire useful evidence, *understanding* to reason about execution traces, *expression* to encode strategic ideas as executable policies, and *generalization* to make revisions useful beyond the cases that prompted them.

Can a coding agent serve as a learner, using game feedback to develop a competitive code policy?

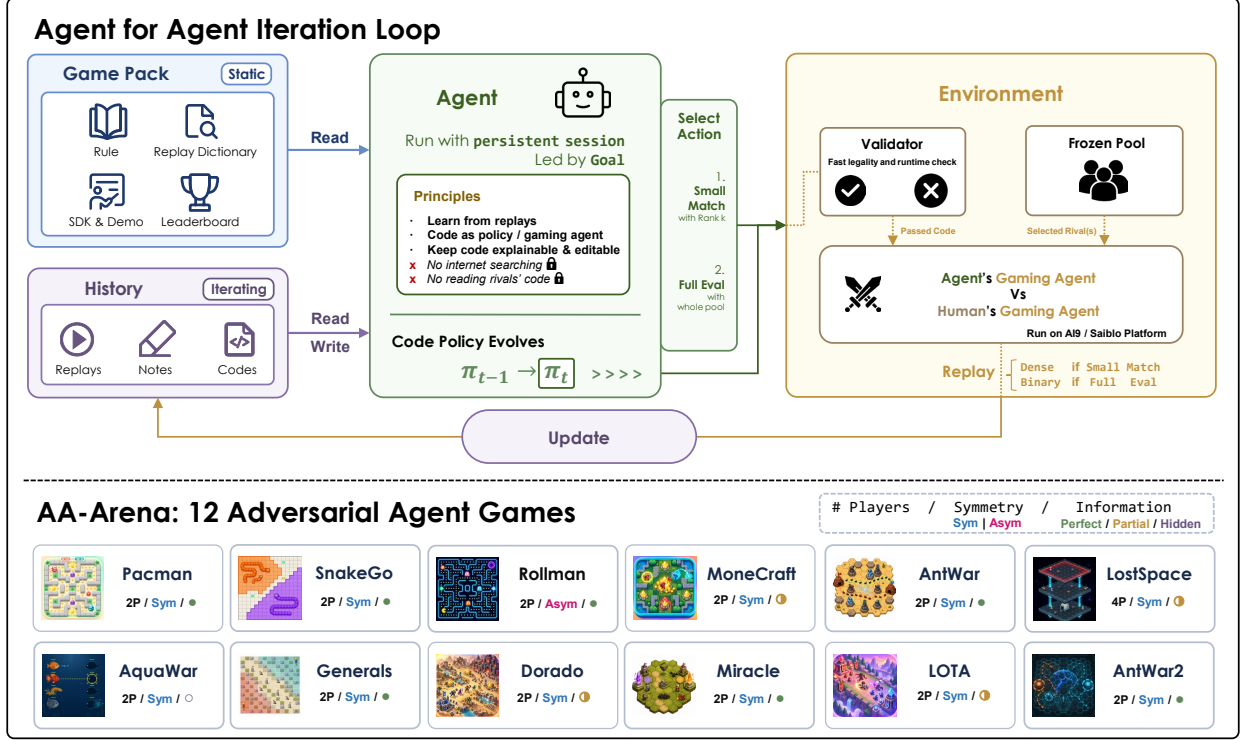


Figure 1 AA-Arena overview. A coding agent uses game resources, replays, and retained notes to revise an executable game-agent policy. Validated snapshots compete against selected opponents in small matches or the frozen pool in full evaluations. Small matches return dense replays; full-pool feedback reports match outcomes together with Elo and rank. The lower panel presents all 12 games, ordered by rule AST node count, with their player counts, role symmetry, and information structure (Table 1).

Weng’s *Learning Beyond Gradients* articulates the related idea of *heuristic learning*: a coding agent reads failures, logs, and replays, then directly improves a program system, accumulating experience in editable software structure (Weng, 2026). Recent work also studies program-policy revision and tournament-based code evolution (Garnier et al., 2026; Wang et al., 2026; Fu et al., 2026; Yang et al., 2026a).

Building on this perspective, we formalize the adversarial-game setting as *Adversarial Heuristic Learning* (AHL). The coding agent selects interactions, interprets feedback, and revises a code policy; the game agent executes that policy against opponents. The coding model’s weights remain fixed. The central question is whether sustained program revision can turn limited match experience into competitive strength across a diverse opponent field.

To evaluate this setting, we introduce *Agents for Agents Arena* (AA-Arena). It brings together 12 original games, runnable backends, and archived human programs from a long-running collegiate competition. Custom rule systems span asymmetric roles, hidden information, resource management, and multi-unit control. Agents obtain detailed replays from selected small matches and Elo and rank from full-pool evaluations. We budget these two feedback channels separately and report the highest officially evaluated score attained within the allowances. Our experiments compare coding models and examine long-horizon revision, opponent selection, feedback use, and learning from off-policy replays.

Our contributions are threefold.

- **We formalize adversarial heuristic learning**, a program-policy optimization paradigm using coding agents to improve executable strategies through adversarial feedback.
- **We introduce AA-Arena**, a benchmark combining real competition games, archived human strategies, and a unified interaction interface with a two-budget evaluation protocol.
- **We evaluate multiple coding models on AA-Arena**, finding that their policies can top some human ladders yet face improvement bottlenecks in complex games, with further experiments revealing the importance of feedback mechanisms and the potential of HL to learn from off-policy data.

2 Adversarial Heuristic Learning

2.1 Preliminaries

Game agent. An adversarial game consists of a shared environment and multiple game agents. Each agent acts on its role’s observation history; the environment responds to their actions and produces an outcome. Games may have more than two players, asymmetric roles, and partially observable states. A game agent executes a *code policy* P implementing the decision rule

$$a \sim \pi_P(\cdot \mid h),$$

where h is the available observation history and a is an action. The submitted program acts autonomously during a match.

Coding agent. A coding agent $\mathcal{A}_\theta$ writes and revises the game agent’s policy. It receives the game material, current source code, and evaluation feedback, and produces an updated executable program. This separation underlies *Agents for Agents*: we evaluate the coding agent through the competitive performance of the game-agent policy it develops.

2.2 Adversarial heuristic learning

Definition 1 (Adversarial heuristic learning). *Adversarial heuristic learning (AHL) is a program-policy optimization paradigm in which a coding agent iteratively revises an executable game-agent policy using adversarial interaction feedback, while its model parameters θ remain fixed.*

Let P_t be the current policy and C_t the retained context. A revision is

$$(P_{t+1}, C_{t+1}) = \mathcal{A}_\theta(P_t, C_t, R, F_t; g),$$

where R is the public game material, F_t is available feedback, and g is the goal of improving competitive performance. Source code, notes, and retained context carry experience forward.

The coding agent explores interactions, interprets their outcomes, and expresses strategy changes in code. Feedback may come from its own policy’s matches or from other agents’ replays. Revisions must reconcile evidence across opponents rather than merely patch a single failure. Section 3 describes the arena and evaluation protocol; workflow details are in Appendix B.1.

3 AA-Arena

3.1 Games and opponent pools

The 12 human-authored competition games cover tactical combat, territory, resource management, and survival, each with a verified, frozen opponent pool (Table 1). Agents receive rules, SDKs, replay documentation, a leaderboard, and a permitted reference program. Appendices H and B describe the games and resources.

Game	Strategic setting	Pool programs	AST nodes	Rules (RA)
Pacman	Competitive maze collection	44	1,127	219
SnakeGo	Snake movement and territory	141	1,538	308
Rollman	Asymmetric maze pursuit	64	1,539	312
MoneCraft	Mining and resource control	112	1,823	347
AntWar	Tower defence and economy	114	2,160	368
LostSpace	Multiplayer survival and escape	111	2,297	426
AquaWar	Hidden-identity tactical combat	170	2,531	439
Generals	Territorial and army control	197	2,683	461
Dorado	Resource competition and army development	323	4,241	616
Miracle	Hex-grid unit tactics	253	4,391	581
LOTA	Hero control and lane combat	200	4,513	684
AntWar2	Ordered tower-defence operations	191	6,368	1,008

Table 1 The 12 games, ordered by increasing rule AST node count. Pool sizes count executable programs, not unique human participants. AST and RA denote rule AST node count and rule count, respectively.

Rule complexity. Kolmogorov complexity $K(R) = \min_{p: U(p)=R} |p|$ is the length of the shortest program describing R on a fixed universal machine U (Vítányi, 2020). We use $C_{\text{AST}}(G) = |\text{AST}(R_G)|$, the AST node count of the game’s rule representation R_G , as a computable description-size proxy, alongside the rule count RA. These are representation-dependent descriptors, not exact Kolmogorov complexities or direct measures of optimal-play difficulty. The same rule-analysis procedure gives *Go* 501 AST nodes and 125 rules, and *chess* 1,551 AST nodes and 238 rules. These provide familiar reference points: the suite spans 1,127–6,368 AST nodes and 219–1,008 rules, covering a broad range of rule-description sizes.

3.2 Interaction interfaces

Small matches let the coding agent select opponents and inspect detailed replays. **Full-pool evaluations** score a submitted policy against the complete opponent pool, returning Elo, rank, and per-opponent outcomes. Both interfaces evaluate immutable program snapshots through the same game referee, and both return feedback that the agent can use for subsequent revisions.

3.3 Evaluation protocol

Each run receives two budgets: B_s small-match units (one per requested opponent) and B_ℓ full-pool evaluations. Default limits are $B_s = 128$ and $B_\ell = 16$. The agent freely interleaves code revisions and feedback requests within these limits.

The final result is the highest officially evaluated Elo attained during the run, together with its corresponding rank. If $\mathcal{V}$ is the set of officially evaluated snapshots, including the common initialization baseline, then

$$\text{Score}(B_s, B_\ell) = \max_{P \in \mathcal{V}} \hat{r}(P).$$

We retain the policy attaining this score. Elo uses fixed opponent ratings; ranking and budget-accounting details, including the baseline’s separate allowance, are given in Appendix B.

4 Experiments

4.1 Main Results

Table 2 reports the retained champion’s Elo and its insertion rank in each game’s frozen opponent ladder. The main protocol provides 128 small-match units and 16 full-pool evaluations; reaching rank 1 terminates a run early. A small-match unit tests one chosen opponent, with the game’s prescribed role assignments, whereas a full-pool evaluation measures a submitted policy against the entire verified pool. The champion is

Table 2 Main results across 12 adversarial games. Each entry gives Elo ($\uparrow$) and rank ($\downarrow$) in the game’s frozen ladder, for the retained champion under a 128-small/16-full evaluation budget. Bold marks the highest Elo among the evaluated models in each row. All models use **max** reasoning effort. Except for Opus, the harness runs each agent in a native persistent Codex session with an explicit goal of improving competitive performance. A dash denotes an unavailable result.

Game	GLM-5.3		Kimi K3		Qwen 3.8		DeepSeek V4 Pro		LongCat 2.0		GPT6-sol		Opus5.5	
	Elo	Rank	Elo	Rank	Elo	Rank	Elo	Rank	Elo	Rank	Elo	Rank	Elo	Rank
Rollman	581.2	1	636.9	1	636.9	1	807.7	1	534.2	1	—	—	—	—
Pacman	2281.6	1	2196.3	2	2236.1	1	2096.8	4	1835.0	10	—	—	—	—
AntWar	1464.2	5	1130.2	9	1355.0	5	1167.2	9	1028.3	11	—	—	—	—
AquaWar	1494.1	1	1494.1	1	1539.7	1	1464.5	1	919.3	71	—	—	—	—
Generals	1441.2	17	1433.3	17	1624.9	10	1340.8	23	1010.3	58	—	—	—	—
LostSpace	1980.1	2	1916.1	2	1797.7	3	1956.8	2	1052.9	77	—	—	—	—
Miracle	1436.8	26	1432.8	26	1500.2	18	1448.8	24	1425.0	27	—	—	—	—
Dorado	2210.3	2	2023.4	16	2035.8	14	2187.8	3	1512.4	156	—	—	—	—
MoneCraft	2359.2	1	2288.2	3	2288.2	3	2288.2	3	2069.9	9	—	—	—	—
LOTA	2147.4	22	2133.2	22	2099.0	25	2140.3	22	1823.3	47	—	—	—	—
SnakeGo	1141.2	4	1165.9	3	1107.5	5	1097.0	5	1153.3	4	—	—	—	—
AntWar2	1977.1	9	1644.2	19	2183.5	6	1511.3	29	1395.9	37	—	—	—	—
Rank-1 games	4		2		3		2		1		—		—	

selected by rank, with Elo breaking ties. Consequently, the reported result is the best officially evaluated policy retained during a run, rather than necessarily the last submitted policy.

GLM-5.3 achieves the highest Elo on six games and reaches rank 1 on *Rollman*, *Pacman*, *AquaWar*, and *MoneCraft*. Qwen 3.8 leads on four games, including *Generals* and *AntWar2*; Kimi K3 leads on *SnakeGo*, and DeepSeek V4 Pro on *Rollman*. Performance therefore depends on the game as well as the coding model. Even the strongest models remain below the leading submitted agents on *Generals*, *Miracle*, and *LOTA*. These gaps motivate an analysis of how executable policies improve, how the models spend their evaluation budgets, and whether additional interaction can close the remaining gap.

4.2 Case Study and Trajectory Patterns

Dorado: repeated gains through difficult iteration. GLM-5.3’s *Dorado* policy develops from a weak competitor into a near-leading agent, improving from Elo 1760.2 (rank 41) to 2210.3 (rank 2). Early corrections to scoring and defence establish a viable policy; a sequence of economic refinements then drives the main climb, and a conditional **counterattack** delivers the final gain. Progress is uneven, with failed revisions interleaved with advances that reshape the policy’s decision logic. This trajectory illustrates Heuristic Learning: the coding agent turns replay evidence into increasingly effective executable heuristics, making *Dorado* an informative case study (Figure 2).

What produces each improvement? Table 3 links each champion improvement to a concise source-code excerpt. The early gains align the policy with **terminal property scoring** and strengthen **base defence**. The main climb from the 8th through the 11th full-pool evaluations corrects the economy: count only **living miners**, discourage **duplicate assignments**, track **mine opening windows**, and concentrate **upgrades on two core heroes**. The **two-hero allocation** delivers the largest adjacent gain (+147.7 Elo), reaching rank 3. After several regressions, conditioning a **one-hero counterattack** on sustained nearby aggression reaches rank 2. Progress therefore comes from correcting specific state interpretations and resource decisions; executable changes alone do not ensure better play.

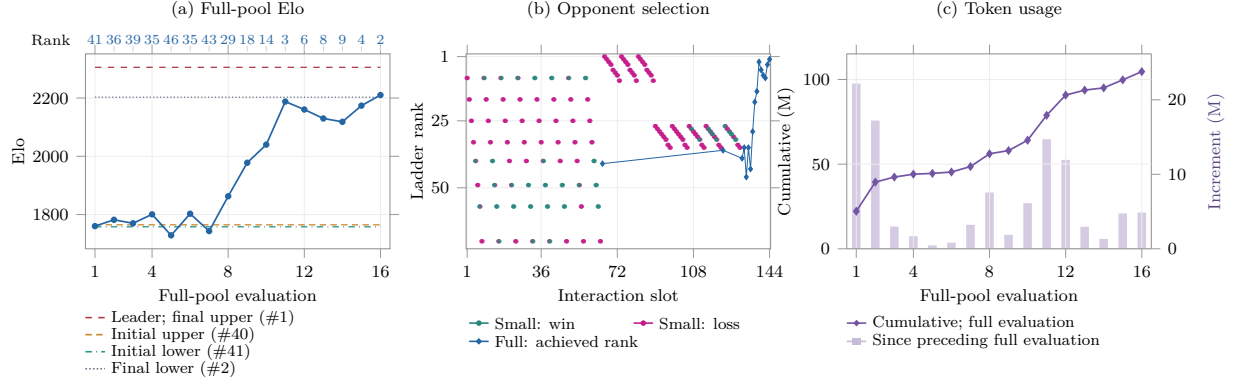


Figure 2 GLM-5.3 on *Dorado*. (a) Full-pool Elo; the aligned row above the curve gives the rank at every evaluation, and reference-line legends give frozen-ladder ranks. (b) Small–full interaction order on 144 equally spaced budget slots (128 small + 16 full). Each opponent test occupies one slot; batches expand in requested-opponent order, with role outcomes slightly offset within that slot. Small-match dots show the selected opponent’s ladder rank, coloured green for wins, magenta for losses, and grey for draws; diamonds show the learner’s achieved rank at full-pool evaluations (rank 1 is at the top). (c) Tokens sampled at the 16 full-pool evaluations (diamonds); bars include all tokens since the preceding full-pool evaluation, with the first bar counting from the run start. The 128 opponent tests comprise 16 batches and 256 role outcomes.

Table 3 *Dorado* champion milestones. Natural-language mechanisms and abridged C++ diff excerpts from `strategy/ai.cpp`. Added (+) and deleted (-) excerpts retain their source line numbers; ellipses omit context. The 6th full-pool evaluation improves Elo at the same rank.

Full-pool eval.	Elo	Rank	Policy adjustment	Code excerpt
2	1782.0	36	Keep mining ; attack only a vulnerable enemy base .	+232 <code>enemyBase != NULL && enemyBase->hp < 1800 && (int)myHeroes.size() >= 4;</code>
4	1800.7	35	Use all living heroes to defend a serious threat.	+245 <code>int count = seriousThreat ? (int)defenders.size() : 2;</code>
6	1802.8	35	Trigger full defence for active threats or recent damage.	+228 <code>const bool seriousThreat = ... baseThreat recentBaseHit;</code>
8	1862.8	29	Exclude dead miners from occupancy; spread assignments.	+343 <code>if (alive && mineUsable(it->second, round)) occ[it->second]++;</code> -477 <code>... + 600LL * occ[m];</code> +472 <code>... + 2000LL * occ[m];</code>
9	1977.9	18	Penalize duplicate mine assignments more strongly.	-472 <code>... + 2000LL * occ[m];</code> +472 <code>... + 5000LL * occ[m];</code>
10	2040.1	14	Track mine opening windows and prioritize open mines .	+217 <code>s_mineExistUntil[i] = round + exist->val[0] - 1;</code> + 36 <code>#define MINE_OPEN_OCCUPANCY_PENALTY 50000LL</code>
11	2187.8	3	Concentrate upgrades on a two-hero core ; preserve mining income .	+296 <code>if (s_dutyId < 0 && (int)s_levledeCore.size() < 2 ...)</code> +316 <code>if (!coreHammerguard && type == "hammerguard") priority = 0;</code>
16	2210.3	2	Allow one-hero counterattacks only after sustained aggression.	+410 <code>s_aggressionRounds = tightAggression ? s_aggressionRounds + 1 : 0;</code> +412 <code>const bool corePush = activeCoreHeroes >= 2 && aggressionThreat;</code>

Coding ability. The *Dorado* snapshots show how a replay-derived hypothesis becomes an executable heuristic: GLM-5.3 corrects **living-miner counts**, tracks **mine availability**, and implements a **temporal threat gate**. Compilation, smoke checks, and custom simulators support these revisions. Yet the **four-hero roster** and unconditional **Sacrifice** also compile while losing Elo. The bottleneck is choosing a strategically useful change and validating its effect, beyond implementing it correctly.

Long-horizon planning across budgets. *Dorado* exhausts all 128 small units before the 3rd full-pool evaluation, leaving 14 full-pool evaluations without fresh small-match replays. Later gains therefore rely on stored evidence and offline reasoning. This allocation separates hypothesis generation from targeted testing: the agent can still measure a revision against the whole pool, but cannot cheaply probe its mechanism against selected opponents. Effective planning must preserve both diagnostic feedback and full-pool validation for later revisions.

In-context understanding of rules and replays. *Dorado*’s gains repeatedly correct the model’s interpretation of the game: a visible **mine** may be closed, a dead **hero** should not occupy a **mining slot**, and distributing **upgrades** across every **hero** need not maximize **terminal property**. Useful learning requires reconstructing state and causal relationships from rules and replays. The decisive improvement is a better account of why a policy succeeds or fails, encoded in the next revision.

Three recurring trajectory patterns. We classify all 12 GLM-5.3 trajectories by the progress of officially evaluated policies, excluding the supplied starter-policy evaluation from the plotted sequence. Success denotes rank-1 early stopping; Progress denotes sustained gains with fluctuations; Stagnation denotes a persistent plateau after initial adaptation. Figures 3–5 show the same three views for every game. Row headings report the retained champion; panel (a) reports each submitted policy’s rank.

- **Success:** *Rollman*, *MoneCraft*, *Pacman*, and *AquaWar* reach rank 1 after 4, 1, 7, and 9 full-pool evaluations, respectively. *Rollman* and *MoneCraft* also leave small-match budget unused (101/128 and 52/128); *Pacman* and *AquaWar* spend all 128 small units but stop before exhausting the full budget.
- **Progress:** *AntWar*, *Dorado*, *LostSpace*, *SnakeGo*, and *AntWar2* achieve progressively stronger champions while individual submissions can regress. *Dorado* shows several consecutive advances in the middle of the run; *AntWar2* has a sharp intermediate collapse; *LostSpace* has fewer advances and a long fluctuating tail. A high final rank can therefore coexist with substantial local instability.
- **Stagnation:** *Generals*, *Miracle*, and *LOTA* enter an extended plateau after their initial adaptation. *Generals* stabilizes around rank 17 after its early climb, *Miracle* repeatedly returns to approximately rank 26–29, and *LOTA*’s best rank remains 22 throughout the full-pool sequence. Additional changes and evaluations mainly revisit similar performance levels.

Overview across the twelve games. The three patterns expose different limits of the same learning loop. Success on *Rollman*, *MoneCraft*, *Pacman*, and *AquaWar* shows that the agent can rapidly connect rules, feedback, and effective code when a useful heuristic is accessible. Progress on *AntWar*, *Dorado*, *LostSpace*, *SnakeGo*, and *AntWar2* shows a slower process of discovering and validating such heuristics; strong final ranks coexist with costly reversals. Stagnation on *Generals*, *Miracle*, and *LOTA* indicates that continued implementation effort alone does not yield better strategic understanding. Budget planning cuts across these categories: several Progress and Stagnation runs exhaust small feedback early, while *SnakeGo* leaves 11 small units unused after spending its full budget. Together, the trajectories distinguish coding competence from the ability to interpret evidence, select productive hypotheses, and sustain their evaluation over a long run.

Figure 3 Success. Each game shows (a) full-pool Elo and ranks, (b) small–full interaction order, and (c) token use. Curves end at rank-1 early stopping. Plot conventions follow Figure 2.



Figure 4 Progress. Best-so-far performance rises despite regressions in individual submissions. Each game shows full-pool Elo and ranks, interaction order, and token use, following Figure 2.

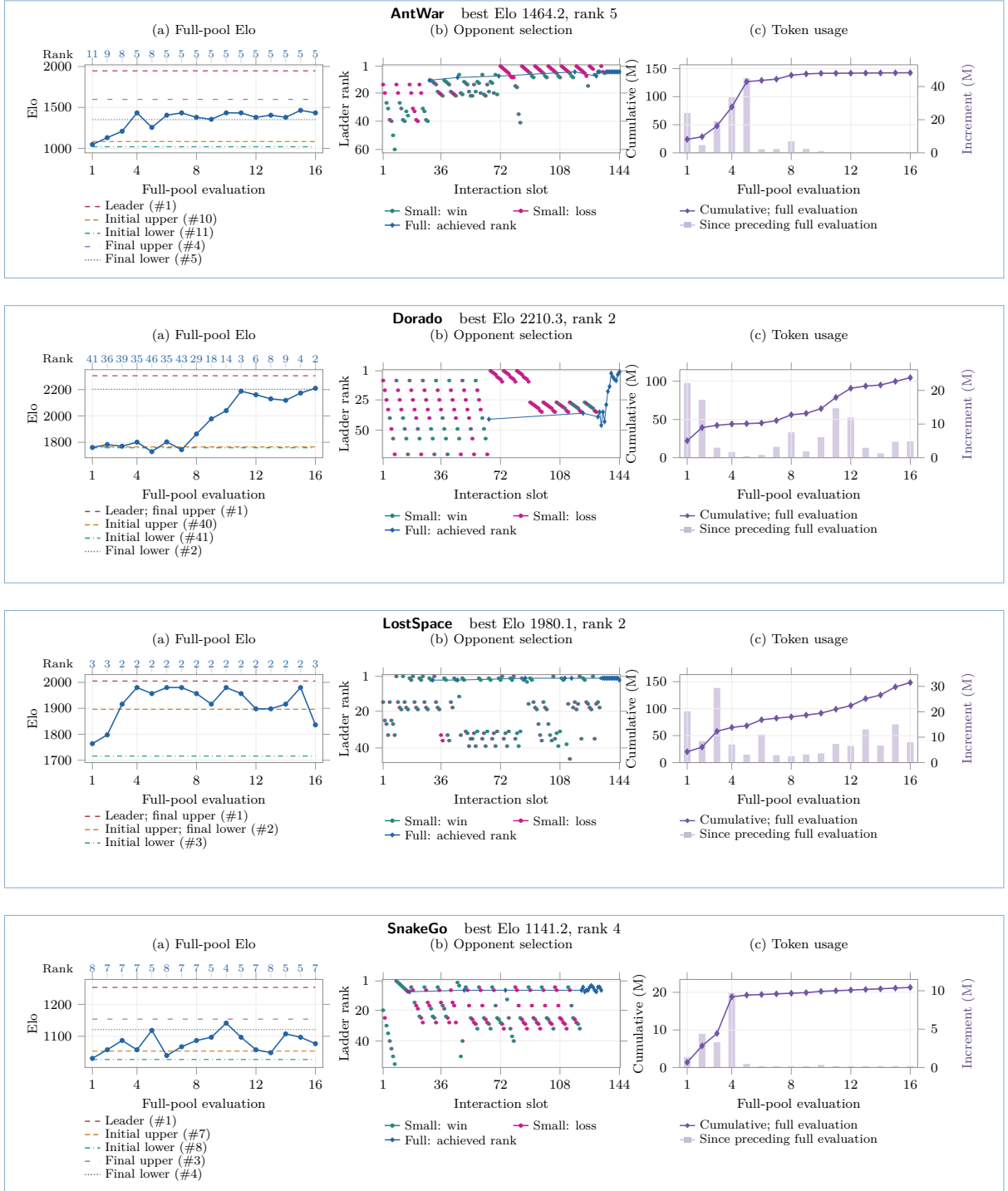


Figure 4 Progress (continued). Full-pool Elo and ranks, interaction order, and token use.

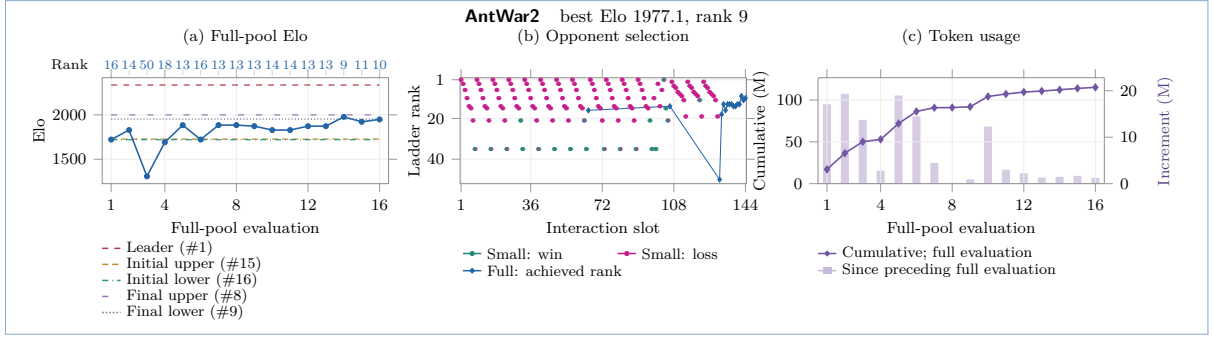
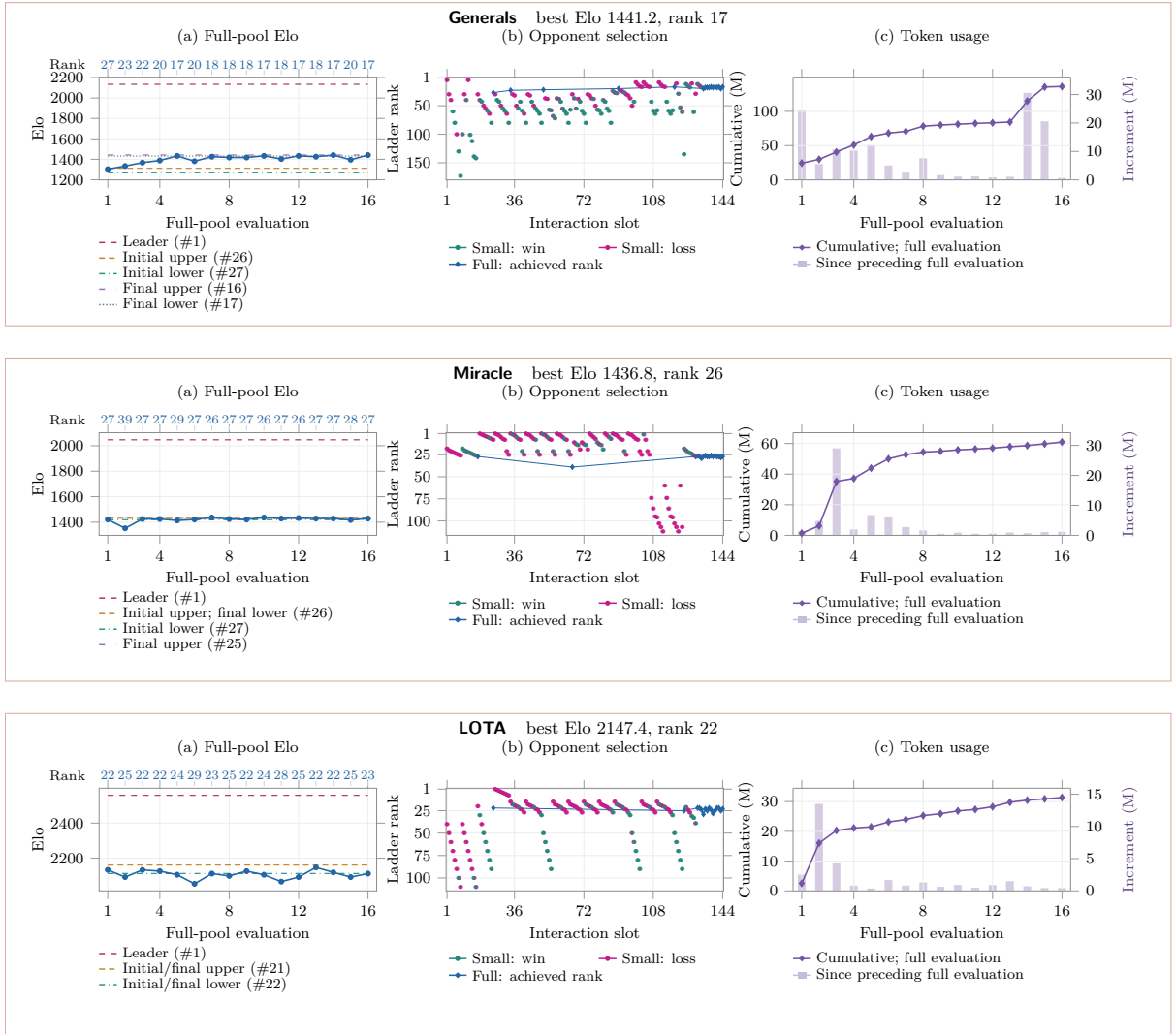


Figure 5 Stagnation. *Generals*, *Miracle*, and *LOTA* show limited sustained improvement after initial adaptation. Small-match feedback is exhausted before 11, 13, and 14 of their 16 full-pool evaluations, respectively. Plot conventions follow Figure 2.



4.3 Have Low-Performing Games Converged?

We continue GLM-5.3 on four games with weak main-stage results: *Generals*, *Miracle*, *LOTA*, and *AntWar2*. Each continuation inherits the policy workspace and the same evaluation ledger, and adds 256 small-match units and 32 full-pool evaluations. The cumulative budget is therefore 384/48, exactly three times the main-stage 128/16 budget. Table 4 compares the retained champions at the two budget levels.

Table 4 Threefold-budget continuation for GLM-5.3. Elo and rank of the retained champion at each budget. The extension adds 256 small units and 32 full-pool evaluations to the same run. A positive Δ Rank indicates an improved position.

Game	128 / 16		384 / 48		Change	
	Elo	Rank	Elo	Rank	Δ Elo	Δ Rank
Generals	1441.2	17	1441.2	17	+0.0	+0
Miracle	1436.8	26	1486.8	19	+50.0	+7
LOTA	2147.4	22	2147.4	22	+0.0	+0
AntWar2	1977.1	9	2117.0	6	+139.8	+3

Generals and *LOTA* show no improvement. *Miracle* gains 50.0 Elo and moves from rank 26 to 19; *AntWar2* gains 139.8 Elo and moves from rank 9 to 6. Thus tripling the interaction budget produces two unchanged results and two limited residual gains. Figure 6 shows the full 48-evaluation trajectories alongside small–full interaction order and token use. Performance remains flat or fluctuates within a narrow range during most of the extension despite continued expenditure. These trajectories and the limited endpoint gains show that the four games have largely converged under this optimization protocol. This supports the main-table budget as an appropriate evaluation allowance: tripling it yields no improvement or only limited additional gains.

4.4 Ablation Studies

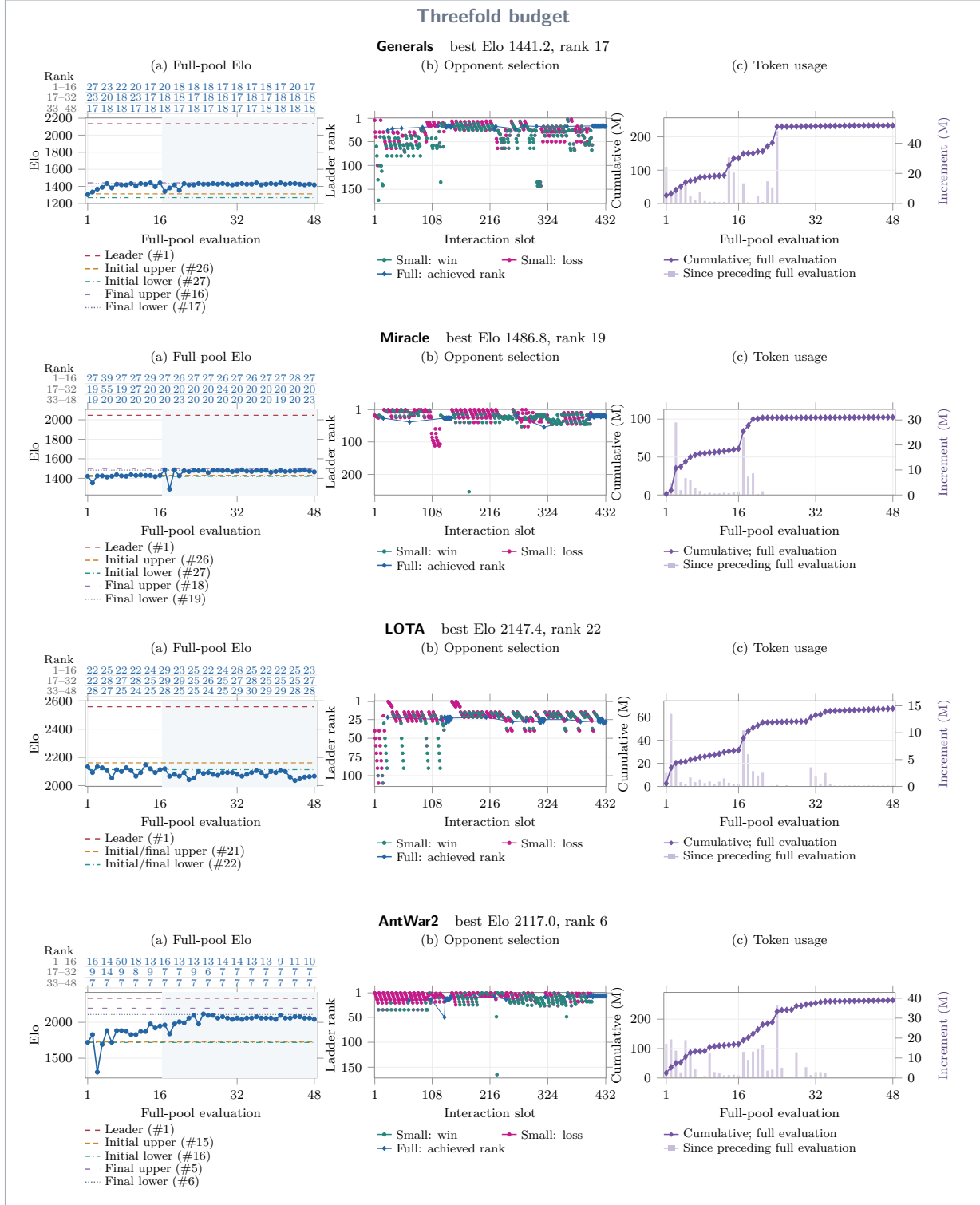
We select one representative game from each trajectory category: *Pacman* (Success), *AntWar* (Progress), and *Miracle* (Stagnation). All ablations use GLM-5.3 with max reasoning effort and the same 128-small/16-full budget. Each axis varies one component while keeping its remaining settings fixed; results are retained-champion Elo, with exact Elo/rank pairs in Appendix C.

4.4.1 Does Opponent Selection Affect Performance?

We compare model-selected opponents, a ladder starting at rank 30, random sampling, and the top four opponents. Small-match batch size is fixed at four and dense feedback is available in all conditions. Figure 7 shows that model selection and the ladder generally outperform random and top-four selection. The ladder leads on *Pacman* and *AntWar*, while model selection leads on *Miracle*. Restricting feedback to the strongest opponents is consistently weakest. The ladder raises opponent strength as the learner improves, keeping challenge difficulty aligned with current ability. Model selection delegates this matching decision to the agent, which can target weaknesses or test a specific hypothesis. Random sampling may spend feedback on uninformative mismatches, while top-four selection can repeatedly expose failures too severe to suggest an attainable next step. Opponent selection therefore shapes a learning curriculum: the most valuable match provides a discernible direction for improvement at the learner’s current level.

4.4.2 Does Dense Replay Feedback Affect Performance?

Dense feedback provides small-match replays and their translations; binary feedback provides only win/non-win outcomes. Both retain conversation history, model-selected opponents, and full-pool feedback, with batch size chosen by the model. Dense feedback improves Elo on all three games: +526.0 on *Pacman*, +238.1 on *AntWar*, and +67.1 on *Miracle* (Figure 8). Dense feedback is a necessary ingredient for strong HL in this setting. A binary outcome identifies failure but gives little evidence about its cause; replays expose the states, actions, and consequences needed to formulate a code-level correction. This richer information channel is a central advantage of the HL interface over a binary-reward RL interface: it lets the agent reason directly



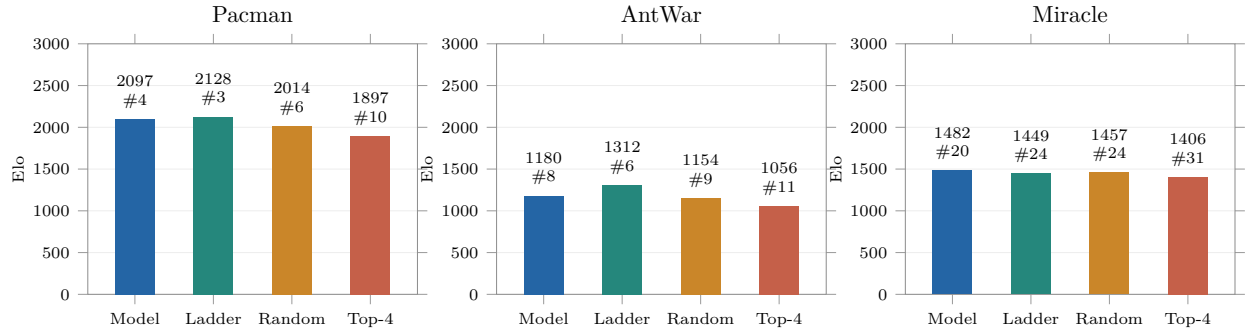


Figure 7 Opponent selection. GLM-5.3 retained-champion Elo with model, ladder, random, and top-four selection; all use dense feedback and batch size 4. Colours distinguish conditions; labels give Elo and rank, and panels share a zero-based Elo axis.

about a failure mechanism and express a targeted revision. The consistent gains across all three trajectory categories show that retaining this diagnostic information is critical to using a limited interaction budget effectively.

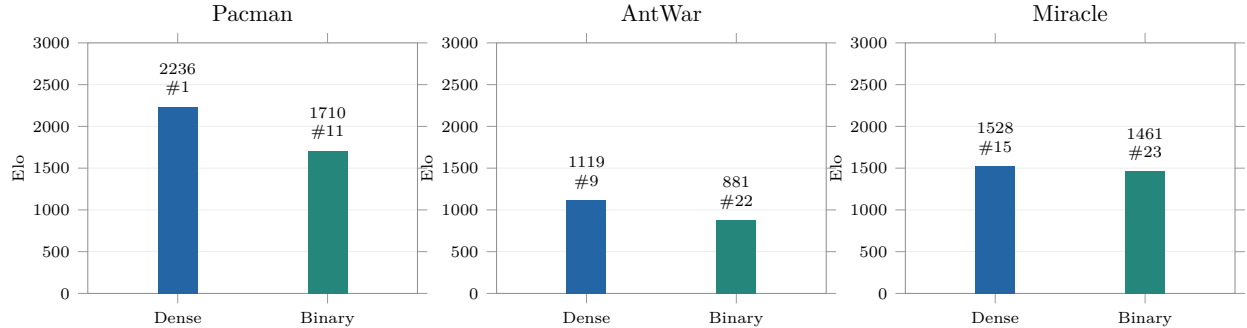


Figure 8 Replay feedback. Dense replays versus binary win/non-win feedback under the same 128/16 budget. Labels give Elo and rank. Dense feedback improves performance in all three representative games.

4.4.3 Does the Number of Opponents per Small Match Matter?

We fix the number of opponents per small-match submission to 1, 2, 4, or 8, using model selection and dense feedback. Intermediate batches perform best: size 4 on *Pacman* and *Miracle*, and size 2 on *AntWar* (Figure 9). A single opponent supplies limited evidence per revision, making it difficult to distinguish a general weakness from an opponent-specific interaction. Larger batches expose a candidate policy to more strategies, but a batch of eight can exhaust the small-match allowance in only 16 full-sized requests. Much of that evidence then concerns early policies, leaving fewer chances to test later revisions even when full-pool evaluations remain available. Intermediate batches of 2–4 provide enough variety to compare failure modes while preserving 32–64 opportunities for targeted testing when the small budget is fully used. The appropriate batch size balances evidence breadth, feedback freshness, and the number of revision cycles; maximizing immediate coverage can undermine long-horizon learning.

4.5 Can Agents Learn from Off-Policy Replays?

We study whether GLM-5.3 can improve its policy by observing games played by other agents. On *Pacman*, *AntWar*, and *Miracle*, the model receives a catalog of dense replays of matches between frozen-pool opponents and autonomously chooses which opponent pair to inspect. It can request up to 128 replay views and interleave them with up to 16 full-pool evaluations of its own policy. The on-policy condition instead obtains replays from its own small matches. Both conditions use max reasoning effort, the same game resources and opponent

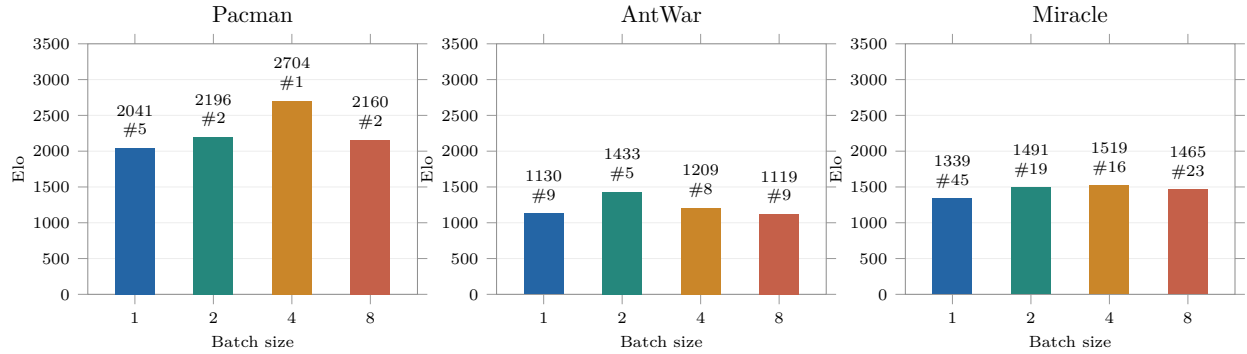


Figure 9 Small-match batch size. Number of opponents per submission, with a fixed total allowance of 128 opponent tests and 16 full-pool evaluations. Labels give Elo and rank. The best size is 4, 2, and 4 for *Pacman*, *AntWar*, and *Miracle*, respectively.

ladders, and the same 128/16 feedback limits; the off-policy condition changes the source of replay evidence while retaining full-pool performance feedback.

Figure 10 shows improvement in all three games. From the first full-pool evaluation to the retained champion, *Pacman* advances from rank 10 to 1, *AntWar* from 41 to 19, and *Miracle* from 27 to 15. *Pacman* reaches rank 1 after 30 replay views and 14 full-pool evaluations. *AntWar* uses 69 views and all 16 evaluations; *Miracle* uses 128 views and 16 evaluations, with 88 views occurring after its final evaluation. These late observations cannot affect its officially evaluated result. The trajectories show that off-policy replays can support policy improvement, while effective allocation of observation and evaluation budgets remains important.

Figure 11 compares the retained champions with the main-table GLM-5.3 runs. The three games illustrate distinct roles for replay evidence.

Pacman is already near the performance ceiling: multiple models reach rank 1 in the main table, and both on-policy and off-policy GLM-5.3 runs reach this target and stop early. Their terminal Elo values (2281.6 and 2404.4) therefore offer little basis for comparing learning quality. Off-policy requires 14 full-pool evaluations, versus 7 on-policy; it attains the same target with more evaluated policy iterations.

On *AntWar*, off-policy reaches Elo 930.7 (rank 19), below on-policy’s 1464.2 (rank 5). Its 69 replay views appear insufficient to recover an effective policy before the 16 full-pool evaluations are exhausted. For this intermediate challenge, on-policy rollouts provide evidence directly tied to the learner’s decisions, making errors easier to locate and its own trajectory easier to correct.

On *Miracle*, off-policy reaches Elo 1528.2 (rank 15), exceeding on-policy’s 1436.8 (rank 26). In this difficult game, weak self-generated rollouts may rarely reveal effective behaviour and thus provide little useful learning signal. Replays of stronger opponents expose strategies that the learner’s own play fails to discover. This result motivates combining external demonstrations with targeted feedback from the learner’s current policy.

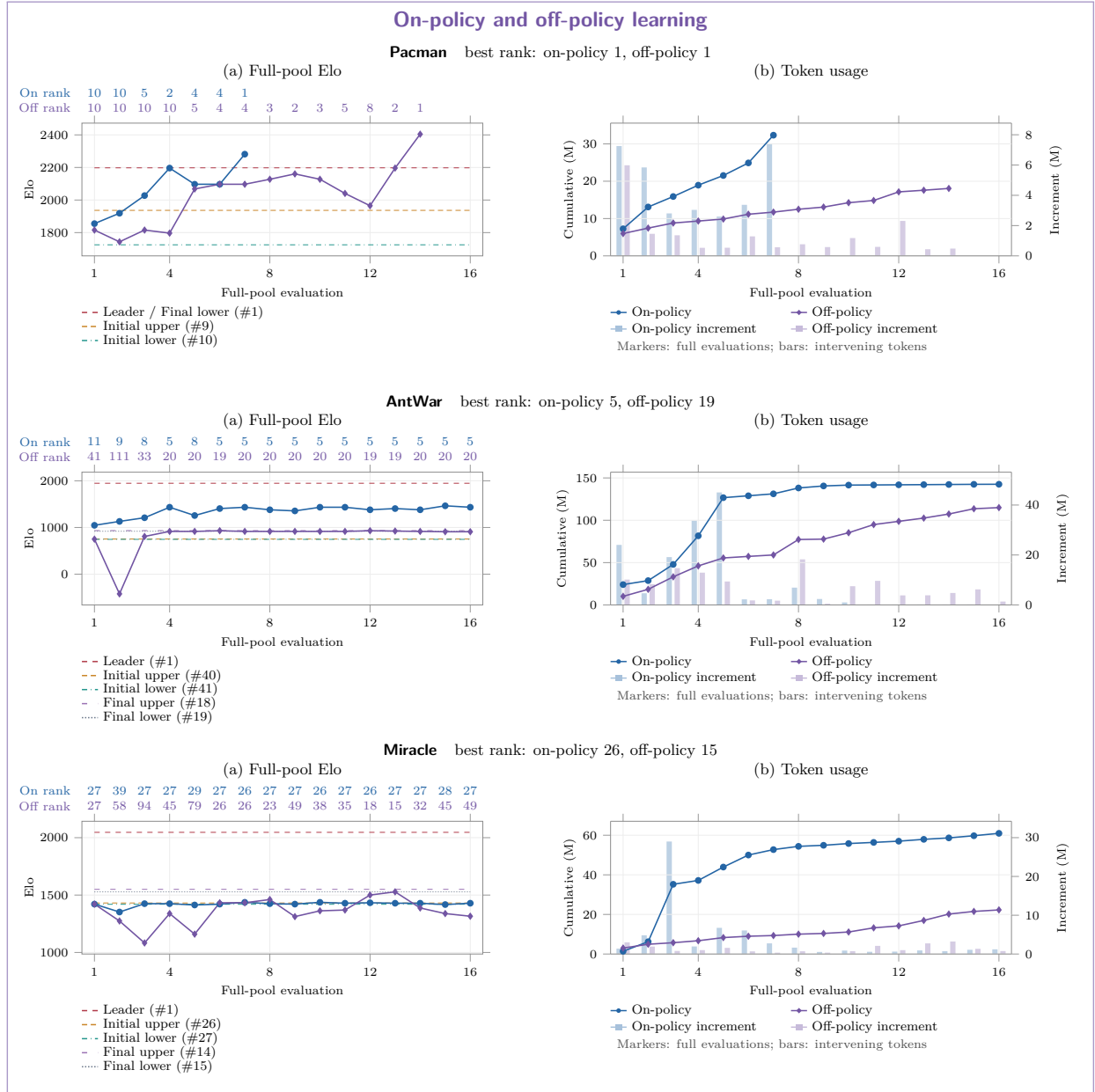


Figure 10 On-policy and off-policy learning trajectories. GLM-5.3 on the same three games; blue circles denote on-policy main-table runs and purple diamonds denote off-policy runs. (a) Full-pool Elo, with aligned rank rows for both conditions. Dashed reference levels and their ladder ranks describe only the off-policy first policy and retained champion. (b) Cumulative tokens at full-pool evaluation submissions (lines) and increments since the preceding full-pool evaluation (paired bars, right axis); the first bars count from the run start. Both panels use full-pool evaluation indices 1–16 and end at the actual evaluation count. Token accounting covers the run start through the final full-pool submission. Both conditions share the same feedback ceilings.

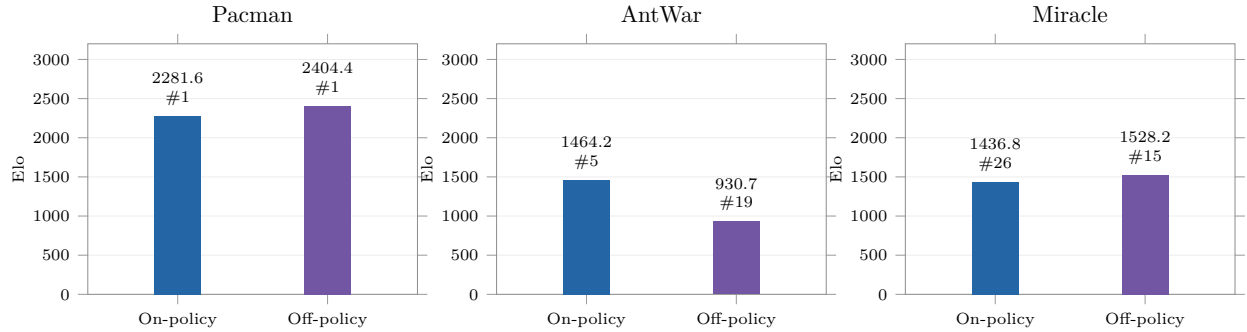


Figure 11 On-policy versus off-policy replay learning. Retained-champion Elo and rank for GLM-5.3 under the same 128/16 budget ceilings. On-policy scores are the main-table results. Labels give Elo and rank; each panel uses the same zero-based Elo scale. Both *Pacman* runs terminate early upon reaching rank 1.

5 Related Work

Table 5 compares six properties: original game design, gameplay diversity, human program pools, iterative self-improvement, freedom to plan the learning process, and long-horizon demands within a game. AA-Arena combines these properties in one evaluation: the agent develops policies for diverse custom games, learns against a complete frozen field of human programs, and controls how to acquire feedback and spend its interaction budgets.

Game-playing benchmarks. BALROG and LMGame-Bench test direct action selection in interactive environments (Paglieri et al., 2025; Hu et al., 2025a). GameBench, GameArena, TextArena, GAMEBoT, ZeroSumEval, and Kaggle Game Arena extend evaluation to competitive reasoning, human interaction, or model tournaments (Costarelli et al., 2024; Hu et al., 2025b; Guertler et al., 2025; Lin et al., 2025; Khan et al., 2025; Kaggle, n.d.). These settings assess the model acting as a player; AA-Arena assesses a coding agent developing the executable player. These suites already cover substantial gameplay diversity and multi-step reasoning. Our distinction is their combination with original competition rules, a reusable human program pool, and an autonomous policy-improvement loop.

Competitive policy development. GACL and ProxyWar evaluate generated game agents, with ProxyWar incorporating test-driven code repair (Game Agent Coding League, n.d.; Peng et al., 2026). The vibe-coding tournament compares generated programs with student programs and includes a limited self-play refinement condition (Danassis and Goel, 2025). CATArena and CodeClash are closer precedents: both repeatedly improve code using tournament feedback (Fu et al., 2026; Yang et al., 2026a). CATArena also exposes peer solutions; CodeClash’s CC:Ladder evaluates progression through fixed human programs. CC:Ladder provides a substantial human program pool, while its opponents and advancement follow a prescribed sequence. AA-Arena instead lets the agent select opponents and interleave targeted matches with full-pool evaluations under separate quotas. This makes evidence selection and budget allocation measurable parts of policy development across 12 custom competition games.

Adversarial games and programmatic policies. Reinforcement learning has become the dominant approach to adversarial games, from value learning and self-play to imperfect-information, multiplayer, and real-time settings (Mnih et al., 2015; Silver et al., 2016, 2017, 2018; Schrittwieser et al., 2020; Zinkevich et al., 2007; Moravčík et al., 2017; Brown and Sandholm, 2018, 2019; Vinyals et al., 2019; OpenAI et al., 2019). Yet opponent pools remain a design choice: high average strength can hide exploitable holes, motivating our frozen pool sampled from a human field rather than a single reference (Lanctot et al., 2017; Wang et al., 2023b; Ke et al., 2026; Nikoleit et al., 2026). Our evaluation instead fixes the feedback interface and compares coding agents against archived human programs. A complementary line represents policies as executable programs, using sketches, distilled trees, program search, or symbolic controllers (Verma et al., 2018; Bastani

Table 5 Game benchmarks and policy-development protocols. ✓: present; ✗: absent; △: partial or restricted to a subset. The six dimensions describe the cited evaluation protocols; definitions appear below.

Benchmark / protocol	Original multi-game	Gameplay diversity	Human code pool	Self-revision	Planning freedom	In-game horizon
Direct game-playing evaluation						
BALROG (Paglieri et al., 2025)	✗	✓	✗	✗	✗	✓
LMGame-Bench (Hu et al., 2025a)	✗	✓	✗	✗	✗	✓
GameBench (Costarelli et al., 2024)	✗	✓	✗	✗	✗	✓
GameArena (Hu et al., 2025b)	△	△	✗	✗	✗	△
TextArena (Guertler et al., 2025)	△	✓	✗	✗	✗	✓
GAMEBoT (Lin et al., 2025)	✗	✓	✗	✗	✗	✓
ZeroSumEval (Khan et al., 2025)	△	✓	✗	△	✗	△
Kaggle Game Arena (Kaggle, n.d.)	✗	✓	✗	✗	✗	✓
Policy development and evaluation						
From Code to Play (Eberhardinger et al., 2025)	✗	✓	✗	✓	✗	✓
Generative code optimization (Kuang et al., 2025)	✗	△	✗	✓	✗	✓
GACL (Game Agent Coding League, n.d.)	✗	✓	✗	✗	✗	✓
ProxyWar (Peng et al., 2026)	✗	✓	✗	△	✗	△
Vibe-coding tournament (Danassis and Goel, 2025)	✗	✗	✓	△	✗	✓
CATArena (game track) (Fu et al., 2026)	△	✓	✗	✓	△	✓
CodeClash (Yang et al., 2026a)	✗	✓	△	✓	△	✓
CC:Ladder (Yang et al., 2026a)	✗	✓	✓	✓	△	✓
EvoPolicyGym (Wang et al., 2026)	✗	✓	✗	✓	✓	✓
AA-Arena (ours)	✓	✓	✓	✓	✓	✓

Original multi-game: multiple independently designed rule systems; mixed suites and variants of established games receive △. **Gameplay diversity:** distinct strategic demands, such as spatial control, resource allocation, hidden information, or social reasoning; closely related tasks receive △. **Human program pool:** a reusable field of executable human contestants covering the evaluated games; isolated or partial comparisons receive △. **Self-improvement:** feedback-driven revision of a persistent strategy or policy across trials; repair-only, prompt-only, or one-round refinement receives △. **Planning freedom:** the developing agent controls feedback selection, timing, and allocation; autonomy within prescribed tournament rounds receives △. **In-game horizon:** actions have consequences across an extended episode requiring multi-step planning or stateful control; short-dialogue or mixed-horizon suites receive △. CC:Ladder’s human pool covers its evaluated ladder games. In-game horizon concerns play within an episode; planning freedom concerns the learning process across episodes.

et al., 2018; Inala et al., 2020; Trivedi et al., 2021; Landajuela et al., 2021); recent work formalizes their size-expressiveness trade-off (Shabadi et al., 2025).

Coding agents and programmatic self-evolution. Language models have changed who writes such programs: they propose policies, search structured spaces, and generate code that evolutionary or learned selectors filter (Liu et al., 2025; Chen et al., 2025; Dat et al., 2024; Hu et al., 2026a,b; Chen et al., 2026a). Other agents write code around a learner, such as reward functions or training procedures (Ma et al., 2024; Lee et al., 2026; Chen et al., 2026b; Chi et al., 2026); AI4AI-Bench is complementary because it rewrites the training algorithm, whereas we maintain the executable policy. Model-controlled program search is the closest representation-level relative (Eberhardinger et al., 2025; Kuang et al., 2025). The common pattern improves executable decision logic from evaluation feedback; AA-Arena studies this process against a human contestant field. Garnier et al. study coding-agent revision of executable controllers in continuous control (Garnier et al., 2026), rather than opponent-relative games, which motivates our narrower adversarial-game scope.

Self-evolving agents and evaluation. Self-evolution can be written into language, prompts, scaffolding, or code (Madaan et al., 2023; Shinn et al., 2023; Zhao et al., 2024; Fernando et al., 2023; Yuksekgonul et al., 2024; Hu et al., 2025c; Romera-Paredes et al., 2023; Zelikman et al., 2024; Wang et al., 2023a; Novikov et al., 2025). Benchmarks increasingly separate transfer from retrieval and the agent from its harness (Shu et al., 2026; Asawa et al., 2026; Ishibashi et al., 2026). Trajectory distillation compresses interaction traces into reusable skills for retrieval and composition (Yang et al., 2026b). Our off-policy experiment asks whether a coding

agent can improve an executable policy using archived games between other agents; we evaluate competitive strength through Elo and rank. EvoPolicyGym is a close protocol precedent: it evaluates repeated policy edits under an agent-allocated episode budget with held-out validation and trajectory diagnostics (Wang et al., 2026). Its single-agent Gym suite and held-out validation differ from our adversarial games and disclosed human opponent field, where both replay feedback and full-pool results guide revision. We measure attained Elo and rank under explicit interface limits, then study the effects of budget, opponent order, and replay access.

6 Future Work

Heuristic learning and reinforcement learning. We aim to formalize agent learning through executable policy revision as heuristic learning and systematically compare it with reinforcement learning. A controlled comparison should examine how the two paradigms represent experience, assign credit, explore policy changes, and generalize, as well as their learning efficiency and attainable performance under matched feedback and compute budgets.

Combining on-policy and off-policy evidence. A hybrid harness could alternate learning from strong opponents with diagnosing the learner’s own rollouts. Off-policy observations would propose promising mechanisms; on-policy tests would reveal where their implementation fails. Repeating this cycle, with budget allocation guided by measured progress, could sustain improvement beyond what either feedback source achieves alone.

Game complexity and capability ceilings. We plan to relate rule-description complexity to the highest performance each model can attain. Building on the AST and rule counts in Table 1, controlled rule variants, multiple seeds, and model and budget sweeps would help distinguish limitations of rule understanding from limitations of search and feedback use, and test how these ceilings change as game complexity increases.

7 Conclusion

We study how coding agents develop and iteratively improve game-agent policies in adversarial games. AA-Arena provides 12 original competition games, fixed human opponent pools, and complementary replay and full-pool feedback. A common Codex-session harness uses a persistent goal to sustain program revision under limits of 128 small-match units and 16 large evaluations, with retained-champion Elo and rank as the primary outcomes.

The model comparison shows that agents can reach the top of some human ladders while remaining far from the strongest programs on others. Across three representative games, GLM-5.3 ablations favour model-selected or ladder opponents, dense replay feedback, and intermediate batch sizes. The opportunity is therefore to improve how an agent selects and uses evidence while maintaining an executable strategy. Off-policy replay learning also improves policies in all three representative games. It reaches the same rank-1 target as on-policy learning on *Pacman* with more policy iterations, trails on *AntWar*, and leads on *Miracle*. This pattern motivates combining external demonstrations with feedback from the learner’s own decisions, particularly when self-generated rollouts provide weak learning signals.

Acknowledgement

We thank the many members of the Student Association of the Department of Computer Science and Technology at Tsinghua University for developing and maintaining the Agent Competition over the years and for supporting this work.

References

- Parth Asawa, Christopher M. Glaze, Gabriel Orlanski, Ramya Ramakrishnan, Benji Xu, Asim Biswal, Vincent Sunn Chen, Frederic Sala, Matei Zaharia, and Joseph E. Gonzalez. Continual learning bench: Evaluating frontier ai systems in real-world stateful environments, 2026. URL <https://arxiv.org/abs/2606.05661>.
- Osbert Bastani, Yewen Pu, and Armando Solar-Lezama. Verifiable reinforcement learning via policy extraction. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018. URL https://proceedings.neurips.cc/paper_files/paper/2018/file/e6d8545daa42d5ced125a4bf747b3688-Paper.pdf.
- Noam Brown and Tuomas Sandholm. Superhuman ai for heads-up no-limit poker: Libratus beats top professionals. *Science*, 359(6374):418–424, January 2018. ISSN 1095-9203. doi: 10.1126/science.aao1733. URL <http://dx.doi.org/10.1126/science.aao1733>.
- Noam Brown and Tuomas Sandholm. Superhuman ai for multiplayer poker. *Science*, 365(6456):885–890, August 2019. ISSN 1095-9203. doi: 10.1126/science.aay2400. URL <http://dx.doi.org/10.1126/science.aay2400>.
- Murray Campbell, A. Joseph Hoane, and Feng-hsiung Hsu. Deep blue. *Artificial Intelligence*, 134(1-2):57–83, January 2002. ISSN 0004-3702. doi: 10.1016/s0004-3702(01)00129-1. URL [http://dx.doi.org/10.1016/S0004-3702\(01\)00129-1](http://dx.doi.org/10.1016/S0004-3702(01)00129-1).
- Minyu Chen, Song Qin, Ling-I Wu, Jianxin Xue, and Guoqiang Li. Teacher-aware evolution of heuristic programs from learned optimization policies, 2026a. URL <https://arxiv.org/abs/2605.10634>.
- Wanyi Chen, Xiao Yang, Xu Yang, Tianming Sha, Qizheng Li, Zhuo Wang, Bowen Xian, Fang Kong, Weiqing Liu, and Jiang Bian. Agent² rl-bench: Can llm agents engineer agentic rl post-training?, 2026b. URL <https://arxiv.org/abs/2604.10547>.
- Zijie Chen, Zhanchao Zhou, Yu Lu, Renjun Xu, Lili Pan, and Zhenzhong Lan. Qube: Enhancing automatic heuristic design via quality-uncertainty balanced evolution, 2025. URL <https://arxiv.org/abs/2412.20694>.
- Yizhe Chi, Wenyi Li, Deyao Hong, Xiaoqiu Wang, Mingju Gao, Kaisen Yang, Bingxiang He, Youjie Zheng, Calvin Xiao, and Qinhuai Na. Ai4ai-bench: Benchmarking llm agents in algorithmic design for recursive self-improvement, 2026. URL <https://arxiv.org/abs/2608.20318>.
- Anthony Costarelli, Mat Allen, Roman Hauksson, Grace Sodunke, Suhas Hariharan, Carlson Cheng, Wenjie Li, Joshua Clymer, and Arjun Yadav. Gamebench: Evaluating strategic reasoning abilities of llm agents, 2024. URL <https://arxiv.org/abs/2406.06613>.
- Rémi Coulom. *Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search*, pages 72–83. Springer Berlin Heidelberg, 2007. ISBN 9783540755388. doi: 10.1007/978-3-540-75538-8_7. URL http://dx.doi.org/10.1007/978-3-540-75538-8_7.
- Panayiotis Danassis and Naman Goel. Can vibe coding beat graduate cs students? an llm vs. human coding tournament on market-driven strategic planning, 2025. URL <https://arxiv.org/abs/2511.20613>.
- Pham Vu Tuan Dat, Long Doan, and Huynh Thi Thanh Binh. Hsevo: Elevating automatic heuristic design with diversity-driven harmony search and genetic algorithm using llms, 2024. URL <https://arxiv.org/abs/2412.14995>.
- Manuel Eberhardinger, James Goodman, Alexander Dockhorn, Diego Perez-Liebana, Raluca D. Gaina, Duygu Çakmak, Setareh Maghsudi, and Simon Lucas. From code to play: Benchmarking program search for games using large language models, 2025. URL <https://arxiv.org/abs/2412.04057>.
- Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution, 2023. URL <https://arxiv.org/abs/2309.16797>.
- Lingyue Fu, Xin Ding, Linyue Pan, Yaoming Zhu, Shao Zhang, Lin Qiu, Weiwen Liu, Weinan Zhang, Xuezhi Cao, Xunliang Cai, Jiaxin Ding, and Yong Yu. Catarena: Evaluating evolutionary capabilities of code agents via iterative tournaments, 2026. URL <https://arxiv.org/abs/2510.26852>.
- Game Agent Coding League. Game Agent Coding League for LLMs. Official benchmark documentation, n.d. URL <https://gameagentcodingleague.com/>. Accessed September 28, 2026; publication date not stated.
- Paul Garnier, Jonathan Viquerat, and Elie Hachem. Heuristic learning for active flow control using coding agents, 2026. URL <https://arxiv.org/abs/2607.11565>.

- Leon Guertler, Bobby Cheng, Simon Yu, Bo Liu, Leshem Choshen, and Cheston Tan. Textarena, 2025. URL <https://arxiv.org/abs/2504.11442>.
- Pablo Hernandez-Leal, Bilal Kartal, and Matthew E. Taylor. A survey and critique of multiagent deep reinforcement learning. *Autonomous Agents and Multi-Agent Systems*, 33(6):750–797, October 2019. ISSN 1573-7454. doi: 10.1007/s10458-019-09421-1. URL <http://dx.doi.org/10.1007/s10458-019-09421-1>.
- Chengpeng Hu, Yingqian Zhang, and Hendrik Baier. Diprl: Learning discrete programmatic policies via architecture entropy regularization, 2026a. URL <https://arxiv.org/abs/2605.18508>.
- Chengpeng Hu, Yingqian Zhang, and Hendrik Baier. Scheduling that speaks: An interpretable programmatic reinforcement learning framework, 2026b. URL <https://arxiv.org/abs/2605.18454>.
- Lanxiang Hu, Mingjia Huo, Yuxuan Zhang, Haoyang Yu, Eric P. Xing, Ion Stoica, Tajana Rosing, Haojian Jin, and Hao Zhang. lmgame-bench: How good are llms at playing games?, 2025a. URL <https://arxiv.org/abs/2505.15146>.
- Lanxiang Hu, Qiyu Li, Anze Xie, Nan Jiang, Ion Stoica, Haojian Jin, and Hao Zhang. Gamearena: Evaluating llm reasoning through live computer games, 2025b. URL <https://arxiv.org/abs/2412.06394>.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems, 2025c. URL <https://arxiv.org/abs/2408.08435>.
- Jeevana Priya Inala, Osbert Bastani, Zenna Tavares, and Armando Solar-Lezama. Synthesizing programmatic policies that inductively generalize. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=S118oANFDH>.
- Yoichi Ishibashi, Taro Yano, and Masafumi Oyamada. Effective harness engineering for algorithm discovery with coding agents, 2026. URL <https://arxiv.org/abs/2605.15221>.
- Kaggle. Game Arena FAQ. Official benchmark documentation, n.d. URL <https://www.kaggle.com/game-arena-faq>. Accessed September 28, 2026; publication date not stated.
- Xinyi Ke, Kai Li, Junliang Xing, Yifan Zhang, and Jian Cheng. Game-theoretic co-evolution for llm-based heuristic discovery, 2026. URL <https://arxiv.org/abs/2601.22896>.
- Haidar Khan, Hisham A. Alyahya, Yazeed Alnumay, M Saiful Bari, and Bülent Yener. Zerosumeval: Scaling llm evaluation with inter-model competition, 2025. URL <https://arxiv.org/abs/2504.12562>.
- Zhiyi Kuang, Ryan Rong, YuCheng Yuan, and Allen Nie. Learning game-playing agents with generative code optimization, 2025. URL <https://arxiv.org/abs/2508.19506>.
- Marc Lanctot, Vinicius Zambaldi, Audrunas Gruslys, Angeliki Lazaridou, Karl Tuyls, Julien Perolat, David Silver, and Thore Graepel. A unified game-theoretic approach to multiagent reinforcement learning. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3323f611e9595c09af38fe67567a9394-Paper.pdf.
- Mikel Landajuela, Brenden K Petersen, Sookyoung Kim, Claudio P Santiago, Ruben Glatt, Nathan Mundhenk, Jacob F Pettit, and Daniel Faissol. Discovering symbolic policies with deep reinforcement learning. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 5979–5989. PMLR, 18–24 Jul 2021. URL <https://proceedings.mlr.press/v139/landajuela21a.html>.
- Hojoon Lee, Ajay Subramanian, Ben Abbatematteo, Vijay Veerabadran, Pedro Matias, Karl Ridgeway, and Nitin Kamra. Rda: Reward design agent for reinforcement learning, 2026. URL <https://arxiv.org/abs/2606.01672>.
- Wenye Lin, Jonathan Roberts, Yunhan Yang, Samuel Albanie, Zongqing Lu, and Kai Han. GAMEBoT: Transparent assessment of LLM reasoning in games. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7656–7682, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.378. URL <https://aclanthology.org/2025.acl-long.378/>.
- Max Liu, Chan-Hung Yu, Wei-Hsu Lee, Cheng-Wei Hung, Yen-Chun Chen, and Shao-Hua Sun. Synthesizing programmatic reinforcement learning policies with large language model guided search, 2025. URL <https://arxiv.org/abs/2405.16450>.

- Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models, 2024. URL <https://arxiv.org/abs/2310.12931>.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhunoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback, 2023. URL <https://arxiv.org/abs/2303.17651>.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharmashan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, February 2015. ISSN 1476-4687. doi: 10.1038/nature14236. URL <http://dx.doi.org/10.1038/nature14236>.
- Matej Moravčík, Martin Schmid, Neil Burch, Viliam Lisý, Dustin Morrill, Nolan Bard, Trevor Davis, Kevin Waugh, Michael Johanson, and Michael Bowling. Deepstack: Expert-level artificial intelligence in heads-up no-limit poker. *Science*, 356(6337):508–513, May 2017. ISSN 1095-9203. doi: 10.1126/science.aam6960. URL <http://dx.doi.org/10.1126/science.aam6960>.
- Henri Nikolait, Ankit Anand, Anurag Murty Naredla, and Heiko Röglin. The art of being difficult: Combining human and ai strengths to find adversarial instances for heuristics, 2026. URL <https://arxiv.org/abs/2601.16849>.
- Alexander Novikov, Ngan Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery, 2025. URL <https://arxiv.org/abs/2506.13131>.
- OpenAI, Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemysław Dębiak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, Rafal Józefowicz, Scott Gray, Catherine Olsson, Jakub Pachocki, Michael Petrov, Henrique P. d. O. Pinto, Jonathan Raiman, Tim Salimans, Jeremy Schlatter, Jonas Schneider, Szymon Sidor, Ilya Sutskever, Jie Tang, Filip Wolski, and Susan Zhang. Dota 2 with large scale deep reinforcement learning, 2019. URL <https://arxiv.org/abs/1912.06680>.
- Davide Paglieri, Bartłomiej Cupiał, Samuel Coward, Ulyana Piterbarg, Maciej Wolczyk, Akbir Khan, Eduardo Pignatelli, Łukasz Kuciński, Lerrel Pinto, Rob Fergus, Jakob Nicolaus Foerster, Jack Parker-Holder, and Tim Rocktäschel. Balrog: Benchmarking agentic llm and vlm reasoning on games, 2025. URL <https://arxiv.org/abs/2411.13543>.
- Wenjun Peng, Xinyu Wang, and Qi Wu. Proxywar: Dynamic assessment of llm code generation in game arenas, 2026. URL <https://arxiv.org/abs/2602.04296>.
- Bernardino Romera-Paredes, Mohammadamin Barekatin, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995): 468–475, December 2023. ISSN 1476-4687. doi: 10.1038/s41586-023-06924-6. URL <http://dx.doi.org/10.1038/s41586-023-06924-6>.
- Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, December 2020. ISSN 1476-4687. doi: 10.1038/s41586-020-03051-4. URL <http://dx.doi.org/10.1038/s41586-020-03051-4>.
- Guruprerana Shabadi, Nathanaël Fijalkow, and Théo Matricon. Programmatic reinforcement learning: Navigating gridworlds, 2025. URL <https://arxiv.org/abs/2402.11650>.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL <https://arxiv.org/abs/2303.11366>.
- Yiheng Shu, Bernal Jiménez Gutiérrez, Saisri Padmaja Jonnalagedda, Yuguang Yao, Huan Sun, and Yu Su. Agentcl: Toward rigorous evaluation of continual learning in language agents, 2026. URL <https://arxiv.org/abs/2606.02461>.
- David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneshelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, January 2016. ISSN 1476-4687. doi: 10.1038/nature16961. URL <http://dx.doi.org/10.1038/nature16961>.

- David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of go without human knowledge. *Nature*, 550(7676):354–359, October 2017. ISSN 1476-4687. doi: 10.1038/nature24270. URL <http://dx.doi.org/10.1038/nature24270>.
- David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharmashan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419): 1140–1144, December 2018. ISSN 1095-9203. doi: 10.1126/science.aar6404. URL <http://dx.doi.org/10.1126/science.aar6404>.
- Dweep Trivedi, Jesse Zhang, Shao-Hua Sun, and Joseph J Lim. Learning to synthesize programs as interpretable and generalizable policies. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 25146–25163. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/file/d37124c4c79f357cb02c655671a432fa-Paper.pdf.
- Abhinav Verma, Vijayaraghavan Murali, Rishabh Singh, Pushmeet Kohli, and Swarat Chaudhuri. Programmatically interpretable reinforcement learning. In Jennifer Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 5045–5054. PMLR, 10–15 Jul 2018. URL <https://proceedings.mlr.press/v80/verma18a.html>.
- Oriol Vinyals, Igor Babuschkin, Wojciech M. Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H. Choi, Richard Powell, Timo Ewalds, Petko Georgiev, Junhyuk Oh, Dan Horgan, Manuel Kroiss, Ivo Danihelka, Aja Huang, Laurent Sifre, Trevor Cai, John P. Agapiou, Max Jaderberg, Alexander S. Vezhnevets, Rémi Leblond, Tobias Pohlen, Valentin Dalibard, David Budden, Yury Sulsky, James Molloy, Tom L. Paine, Caglar Gulcehre, Ziyu Wang, Tobias Pfaff, Yuhuai Wu, Roman Ring, Dani Yogatama, Dario Wünsch, Katrina McKinney, Oliver Smith, Tom Schaul, Timothy Lillicrap, Koray Kavukcuoglu, Demis Hassabis, Chris Apps, and David Silver. Grandmaster level in starcraft ii using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, October 2019. ISSN 1476-4687. doi: 10.1038/s41586-019-1724-z. URL <http://dx.doi.org/10.1038/s41586-019-1724-z>.
- Paul M.B. Vitányi. How incomputable is kolmogorov complexity? *Entropy*, 22(4):408, April 2020. ISSN 1099-4300. doi: 10.3390/e22040408. URL <http://dx.doi.org/10.3390/e22040408>.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023a. URL <https://arxiv.org/abs/2305.16291>.
- Tony Tong Wang, Adam Gleave, Tom Tseng, Kellin Pelrine, Nora Belrose, Joseph Miller, Michael D Dennis, Yawen Duan, Viktor Pogrebnik, Sergey Levine, and Stuart Russell. Adversarial policies beat superhuman go AIs. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 35655–35739. PMLR, 23–29 Jul 2023b. URL <https://proceedings.mlr.press/v202/wang23g.html>.
- Zhilin Wang, Han Song, Runzhe Zhan, Jusen Du, Jiacheng Chen, Tianle Li, Qingyu Yin, Yulun Wu, Zhennan Shen, Tong Zhu, Yanshu Li, Guanjie Chen, Derek F. Wong, Yafu Li, Yu Cheng, and Yang Yang. Evopolicygym: Evaluating autonomous policy evolution in interactive environments, 2026. URL <https://arxiv.org/abs/2607.02440>.
- Jiayi Weng. Learning beyond gradients. <https://trinkle23897.github.io/learning-beyond-gradients/>, May 2026. Blog post.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024. URL <https://arxiv.org/abs/2404.07972>.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering, 2024. URL <https://arxiv.org/abs/2405.15793>.
- John Yang, Kilian Lieret, Joyce Yang, Carlos E. Jimenez, Muhtasham Oblokulov, Aryan Siddiqui, Ofir Press, Ludwig Schmidt, and Diyi Yang. Codeclash: Benchmarking goal-oriented software engineering, 2026a. URL <https://arxiv.org/abs/2511.00839>.
- Kaisen Yang, Zheng Jiang, Yuzhao Peng, Houde Qian, Boshi Zhang, Youjie Zheng, Shijin Hong, Qingle Liu, Ruoyu

- Han, Bohan Lyu, Bingxiang He, Eren Cai, Calvin Xiao, and Qinhuai Na. Scalable behaviour cloning on browser using via skill distillation, 2026b. URL <https://arxiv.org/abs/2606.32014>.
- Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. Textgrad: Automatic "differentiation" via text, 2024. URL <https://arxiv.org/abs/2406.07496>.
- Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (stop): Recursively self-improving code generation, 2024. URL <https://arxiv.org/abs/2310.02304>.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(17):19632–19642, March 2024. ISSN 2159-5399. doi: 10.1609/aaai.v38i17.29936. URL <http://dx.doi.org/10.1609/aaai.v38i17.29936>.
- Martin Zinkevich, Michael Johanson, Michael Bowling, and Carmelo Piccione. Regret minimization in games with incomplete information. In J. Platt, D. Koller, Y. Singer, and S. Roweis, editors, *Advances in Neural Information Processing Systems*, volume 20. Curran Associates, Inc., 2007. URL https://proceedings.neurips.cc/paper_files/paper/2007/file/08d98638c6fcd194a4b1e6992063e944-Paper.pdf.

A Snapshot provenance and reproducibility

The main results and GLM-5.3 trajectories use the September 24, 2026 main-table bundle. The four threefold-budget experiments use the accompanying GLM-5.3 384/48 continuation bundle. The compact companion directory `data/experiments-20260924/` contains the 60 main-stage results, continuation endpoints, submission-level plotting data, and a source-snapshot map for the *Dorado* case. Source hashes and run identifiers support offline verification. The opponent-selection, replay-feedback, and batch-size ablations use the September 22 three-axis bundle for *Pacman*, *AntWar*, and *Miracle*. The off-policy trajectories use the September 26 dense-replay bundle; `data/offpolicy-20260926/` retains their compact source records, results, and timing audit. Their game-resource, leaderboard, opponent-pool, and evaluator hashes match the corresponding main-table runs. Active-match cloning resource measurements use the September 17 cohort.

Interaction panels use equally spaced budget slots, independent of elapsed time. Each small-match opponent consumes one slot, and each full-pool evaluation consumes one slot. Within a batch, opponents follow the recorded request order; this ordering does not imply sequential execution. Small-match points use the selected opponent’s rank, with colour indicating the role outcome and a slight horizontal offset separating prescribed roles within a slot. Large-evaluation diamonds use the learner’s achieved rank. Rank 1 is at the top. Main-stage axes span 144 slots and continuation axes span 432; unused budget remains blank. Client logs cannot separate all API waiting from model processing, so these panels do not estimate inference duration. Token curves sample cumulative input-plus-output counters across sessions at each large-evaluation submission. The horizontal axis is 1–16 for main runs and 1–48 for continuations; early-stopped runs end at their actual evaluation count. Bars give differences between consecutive large-evaluation counters, including all intervening small-match-related work; the first bar counts from the run start. Post-final-submission token traffic is not plotted. Cached input is part of input, not an additional charge. Starter-policy evaluations are excluded from the full-pool sequence.

The scripts `build_result_tables.py` and `build_experiment_figures.py` reproduce the tables and PGFPlots sources from the compact data. Full reevaluation additionally requires the frozen game resources, opponent programs, and runtime dependencies. The resource, opponent-pool, and leaderboard hashes identify those inputs. The measured-stage initialization of GLM-5.3 *LostSpace* and Qwen 3.8 *AquaWar* includes an existing policy and replay workspace.

B Arena resources and evaluation details

Public resources and opponents. Each opponent is a versioned executable program backed by a frozen manifest. Pool membership, opponent ratings, and resources are fixed for each run. Related versions and incomplete entrant identities mean pool counts are not counts of independent human participants. Public resources include rules, SDKs, replay-format documentation, the leaderboard, and the permitted rank-40 reference example. Other opponent source code and evaluator internals are outside the workspace.

Execution and budget accounting. Small requests may batch up to 8 opponent IDs, charging one unit per opponent; a full-pool request costs one large-evaluation unit. Role expansion follows the game’s referee contract and does not change these units. Player errors are adjudicated by the game, while infrastructure failures are recorded separately. Accepted tasks and charges remain in the ledger during recovery to avoid double charging. Pending evaluations are not treated as measured scores.

The budgets limit feedback access, not source edits, model calls, tokens, or wall-clock time. Runs may stop at first place or the full-pool call limit, leaving unused allowance. The controller retains the best completed snapshot by rank, then Elo. With fixed opponent ratings, this is equivalent to maximizing Elo. The initial controller-evaluated baseline is outside the adaptive budgets and is eligible for retention. The four continuation runs add 256 small units and 32 full-pool evaluations to the same ledger, for cumulative 384/48 limits. Active-match cloning runs omit the baseline and follow their own stopping rule (Appendix F).

B.1 Coding-agent workflow

Each model–game run uses a persistent Codex workflow with a writable workspace. Retained context and workspace artifacts support resumed turns, session recovery, and context compaction. The coding agent decides when to inspect or edit its program, request small matches against selected opponents, or submit a snapshot for full-pool evaluation. Small matches provide decision-level replays; full-pool evaluations provide Elo, rank, and opponent-level outcomes. Both channels guide subsequent revisions.

The controller tracks the two feedback budgets and retains the best evaluated snapshot by rank, then Elo. The working policy may continue changing even when a revision fails to improve that champion. The output is therefore the strongest retained submission, not necessarily the last program edited.

B.2 Leaderboard scores

Candidate Elo is estimated with a fixed-pool Bradley–Terry model. Let H denote the frozen opponent pool and r_j be a frozen opponent rating and $s_m \in [0, 1]$ the candidate’s score in a completed role-expanded match against opponent $j(m)$. The expected score at candidate rating r is

$$p(r, r_j) = \frac{1}{1 + 10^{(r_j - r)/400}}.$$

The estimator solves

$$\sum_m p(\hat{r}, r_{j(m)}) + p(\hat{r}, \bar{r}_H) = \sum_m s_m + \frac{1}{2}, \quad \bar{r}_H = \frac{1}{|H|} \sum_{j \in H} r_j.$$

The additional term represents a single virtual draw at the full pool’s mean rating, which keeps the estimate finite for perfect records. Opponent ratings remain fixed. The candidate’s rank is $1 + |\{j \in H : r_j > \hat{r}\}|$. The referee determines the scores; in win/draw/loss matches they are respectively 1, $\frac{1}{2}$, and 0.

We report the retained champion’s Elo and rank together because they answer related but different questions: the former summarizes strength on the pool’s rating scale, while the latter gives its position in that field. Elo scales and pool sizes differ across games, so raw Elo values are not averaged into a cross-game score. Since the agent receives large-evaluation results during optimization, these measurements quantify attained strength on the disclosed field; they do not establish generalization to unseen opponents.

C Detailed ablation results

Tables 6–8 report Elo and rank for the three representative games. All conditions are complete. Opponent-selection runs fix batch size 4 and match-base seed 43; feedback and batch-size runs use match-base seed 42. Each axis uses its own matched conditions.

Game	Model	Ladder	Random	Top-4
Pacman	2096.8 / 4	2127.5 / 3	2014.4 / 6	1897.0 / 10
AntWar	1180.5 / 8	1311.7 / 6	1154.4 / 9	1056.2 / 11
Miracle	1482.4 / 20	1448.8 / 24	1457.0 / 24	1406.0 / 31

Table 6 Opponent selection ablation with GLM-5.3. Each cell is Elo / rank under the 128/16 budget.

Game	Dense	Binary
Pacman	2236.1 / 1	1710.1 / 11
AntWar	1118.7 / 9	880.6 / 22
Miracle	1528.2 / 15	1461.1 / 23

Table 7 Replay feedback ablation with GLM-5.3. Each cell is Elo / rank under the 128/16 budget.

Game	1	2	4	8
Pacman	2040.6 / 5	2196.3 / 2	2703.9 / 1	2160.4 / 2
AntWar	1130.2 / 9	1433.4 / 5	1208.7 / 8	1118.7 / 9
Miracle	1338.7 / 45	1491.2 / 19	1518.7 / 16	1465.3 / 23

Table 8 Small-match batch size ablation with GLM-5.3. Each cell is Elo / rank under the 128/16 budget.

Model	Completed	Rank-1 games	Tokens (M)
GLM-5.3	12/12	4	900.78
Kimi K3	12/12	2	549.70
Qwen 3.8	12/12	3	1454.07
DeepSeek V4 Pro	12/12	2	852.21
LongCat 2.0	12/12	1	454.33
GPT5.6sol	0/12	—	—
Opus5	0/12	—	—

Table 9 Main-stage completion and reported token use, summed over the per-game totals. Cached input is included once. Continuations and ablations are excluded; the totals do not measure monetary cost.

Game	Target #5	Target #15	Target #25	Target #35
Rollman	25.86	6.62	8.85	9.47
Pacman	28.77	10.96	10.10	22.99
AntWar	34.12	20.35	13.03	7.34
SnakeGo	18.80	20.96	17.32	15.38
AquaWar	27.18	14.75	27.07	33.30
Generals	24.55	10.93	12.33	41.18
LostSpace	14.51	56.34	26.53	32.26
Miracle	24.93	29.12	19.42	16.39
AntWar2	35.35	21.08	48.39	21.62
MoneCraft	14.12	24.95	10.34	13.85
LOTA	19.47	19.82	23.70	40.14
Dorado	24.05	17.84	14.52	36.74

Table 10 Behavior-cloning data collection with GLM-5.3. Each entry is token use in millions for one independent run; all 48 runs completed 32 small-match units with distinct training seeds, no large evaluations, and a final adaptation after the last replay. These are completion and resource measurements, not imitation-accuracy scores.

D Model resource use

Token accounting includes input and output. Cached input is included in input and must not be added a second time; token totals therefore cannot be read as uncached billing cost. The model totals in Table 9 cover only the default phase. Per-game input, cached-input, output, small-match use, large-call use, and completion state remain available in the companion CSVs. Ablation results are single runs per game and condition; they are not repetitions of the main-table cells.

E Continuation records

The four GLM-5.3 continuation runs consume all 384 small units and 48 full-pool evaluations. Table 4 reports their main-stage and cumulative champions. Figure 6 reports their complete Elo, opponent-selection, and token trajectories.

F Active-match cloning resource use

These 48 GLM-5.3 runs collected replays through active matches against one target, using 32 small-match units and no full-pool evaluations. They completed policy adaptation, but no fidelity scores are available. Table 10 reports token consumption only. Section 4.5 evaluates competitive performance after learning from fixed opponent-pair replays; neither measurement reports action-level imitation fidelity.

G A worked case: *AntWar2*

AntWar2 is a tower-defence game on a 19×19 offset hex grid. Each side begins with a **base** holding 50 **hit points**. Players submit **ordered operation lists** involving **towers**, **upgrades**, **downgrades**, **base development**, and 4 **super-weapons**. Ant movement and combat are advanced by the environment, so the policy shapes economy and defensive coverage rather than steering each **ant** directly.

Order is part of the decision problem. An operation is checked against the state after the accepted prefix of the same round; buying or selling an asset can therefore affect the legality and value of the next operation. A program that chooses reasonable actions individually may still produce a poor sequence. This makes the game a useful case for studying how an agent turns trace evidence into a concrete control-flow revision.

A replay can expose rounds, operations, state changes, resource levels, **towers**, **ants**, and weapon effects through the game’s bounded public format. The reading guide explains how to reconstruct state from deltas

and align it with submitted actions. The model must derive its explanation from these records rather than assume that a single event is a complete state snapshot. Small matches can test an ordering hypothesis against one opponent; a large evaluation then reveals whether the resulting code improves placement across the full human field.

H The evaluated games

The suite shares an evaluation interface, but its policies solve different decision problems. The descriptions below refer to the public rule and SDK bundles used in the experiments; pool sizes are listed in Table 1.

Pacman. *Pacman* is a separate, symmetric competitive maze game. Players collect ordinary and **high-value beans**, avoid **monsters**, and place **mines** that affect movement choices. Collection, collision avoidance, and disruption of the opponent jointly determine the score. Its rules and opponent pool are distinct from *Rollman*’s asymmetric contest.

SnakeGo. *SnakeGo* is a territory game in which **snakes** move, collect items, claim **cells**, and can split or use special actions. Local collision avoidance interacts with longer-horizon territory growth, making a safe immediate move potentially costly at settlement.

Rollman. *Rollman* is an asymmetric maze game. One side collects **beans** and uses temporary skills; the other coordinates **ghosts** to intercept it. The roles differ in both action space and objective, making movement planning and coordinated pursuit distinct policy problems.

MoneCraft. *MoneCraft* is a mining and resource-control game on a symmetric map. Players capture **mines** that generate **currency** and can teleport, place **mines**, or purchase protection. The policy trades movement and disruption against sustained resource production; end-game **currency** and **mine ownership** determine the result under the referee’s rules.

AntWar. *AntWar* combines tower defence with economic management. Players build and upgrade **towers**, improve their **bases**, and time **super-weapons**, while **ants** follow environment-driven paths. The policy must balance present defence with spending that changes future pressure.

LostSpace. *LostSpace* is a multiplayer survival-and-escape game on a layered map that contracts over time. Players collect **supplies** and **keys**, use **tools**, fight or evade rivals, and seek escape. Ranked outcomes create trade-offs between immediate combat, survival, and the timing of an escape attempt.

AquaWar. *AquaWar* is a hidden-identity tactical game. Players select **fish**, make assertions about hidden opponent identities, and choose attacks or type-specific skills. Roster design, inference, and information-sensitive timing all affect the outcome.

Generals. *Generals* combines territorial expansion, army movement, neutral assets, production, technology, and tactical abilities. An effective policy coordinates economic development with the timing and direction of attacks rather than optimizing either in isolation.

Dorado. *Dorado* combines resource competition with **hero** and army development. Players choose **heroes**, produce and command **units**, mine neutral resources, and invest in **upgrades**. Movement, combat, harvesting, and abilities must be coordinated to threaten opposing **bases** while maintaining the resources needed to continue production.

Miracle. *Miracle* is a hex-grid strategy game with heterogeneous **units** and an initial drafting stage. A player selects **creatures** and an **artifact**, then summons, moves, attacks, and uses abilities to destroy the opponent’s **miracle**. Resource limits and unit interactions make tactical coordination central to the policy.

LOTA. *LOTA* is a lane-based combat game. Each player chooses a **hero** and controls movement, attacks, abilities, skill development, and **revival** while **units** advance along fixed **lanes**. Victory depends on destroying the opposing **base** or the referee’s settlement criteria at the turn limit. Effective control couples immediate fights with sustained pressure on defensive structures.

AntWar2. *AntWar2* extends tower-defence strategy with **ordered operation lists** whose accepted prefix changes subsequent decisions. The agent must coordinate **tower placement**, economic development, and weapon timing while respecting sequence-dependent legality. Appendix [G](#) gives the worked example.